

PlantCare AI: Advanced Plant Disease Detection Using Transfer Learning

Project Description:

PlantCare AI aims to develop an accurate and efficient model for classifying plant diseases by employing transfer learning techniques. Utilizing the New Plant Diseases Dataset containing over 87,000 annotated plant leaf images categorized into 38 distinct classes including various diseases affecting crops like Apple, Corn, Grape, Potato, Tomato, and others, the project leverages the pre-trained MobileNetV2 convolutional neural network (CNN) to expedite training and improve classification accuracy. Transfer learning allows the model to benefit from pre-existing knowledge of image features, significantly enhancing its performance and reducing computational costs. This approach provides a reliable and scalable tool for farmers, agricultural professionals, and gardeners, ensuring precise and efficient plant disease identification.

Application Scenarios

Scenario 1: Automated Agricultural Monitoring Systems

Integrating PlantCare AI into automated monitoring systems on farms can revolutionize crop health management. By using transfer learning, the system quickly adapts to the specifics of plant disease classification, capturing images of plant leaves, classifying diseases in real-time, and generating detailed reports. This automation reduces the manual workload on agricultural experts, speeds up diagnostic processes, and ensures high accuracy in results, ultimately improving crop yields and reducing losses due to disease.

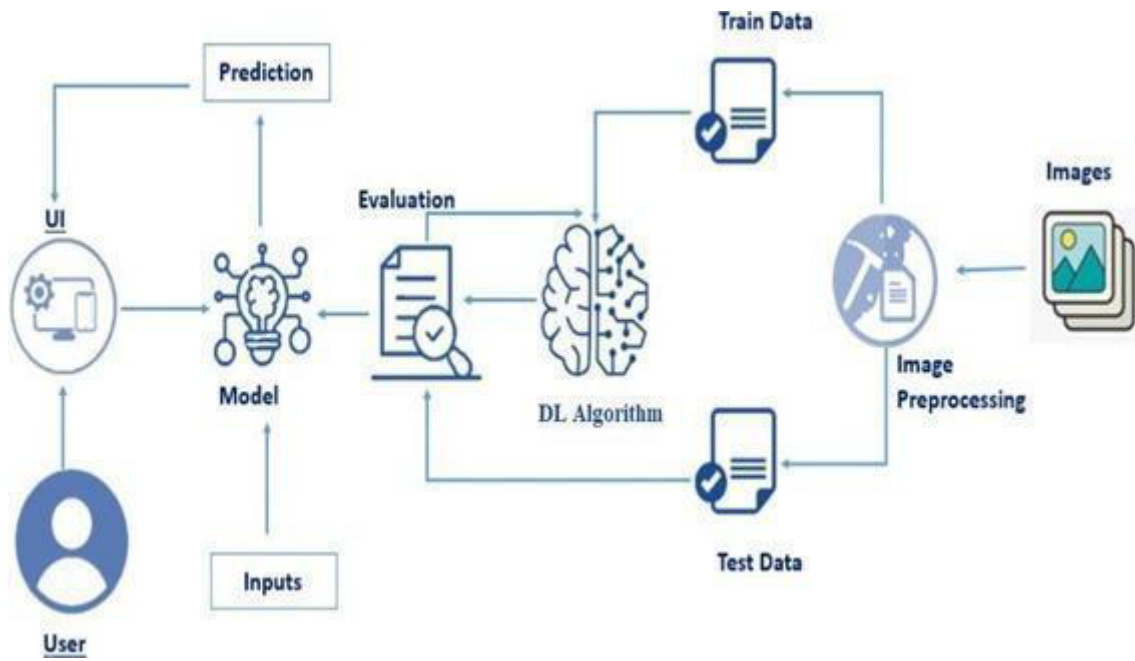
Scenario 2: Mobile Applications for Home Gardeners

PlantCare AI can be deployed as a mobile application to help home gardeners and hobbyists identify plant diseases. With transfer learning, the model's ability to accurately classify diseases from diverse plant images is improved, allowing users to simply take photos of affected leaves for instant analysis. This enables timely intervention without the need for expert consultation, making plant care more accessible and helping maintain healthy gardens.

Scenario 3: Educational Tools for Agricultural Training

PlantCare AI's transfer learning-based classification model can be integrated into educational platforms for agricultural training. By incorporating this advanced technology into interactive learning tools, students and agricultural technicians can upload and analyze plant images to receive instant feedback. This hands-on learning experience enhances their understanding of plant pathology and disease identification, providing practical skills crucial for modern agricultural practices.

Technical Diagram:



Prerequisites:

To complete this project, you must require the following software, concepts, and packages

• Anaconda Navigator:

- o Refer to the link below to download Anaconda Navigator

• Python packages:

- o Open anaconda prompt as administrator
- o Type "pip install numpy" and click enter.
- o Type "pip install pandas" and click enter.
- o Type "pip install scikit-learn" and click enter.
- o Type "pip install matplotlib" and click enter.
- o Type "pip install scipy" and click enter.

- o Type "pip install seaborn" and click enter.
- o Type "pip install tensorflow" and click enter.
- o Type "pip install Flask" and click enter.

Prior Knowledge:

You must have prior knowledge of the following topics to complete this project.

- **DL Concepts**

- **Neural Networks:**

- https://www.analyticsvidhya.com/blog/2024/05/ann-vs-cnn-vs-rnn/?utm_source=chatgpt.com

- **Deep Learning Frameworks:**

- https://www.analyticsvidhya.com/blog/2019/03/deep-learning-frameworks-comparison/?utm_source=chatgpt.com

- **Transfer Learning:**

- https://www.datacamp.com/blog/what-is-transfer-learning-in-ai-an-introductory-guide?utm_source=chatgpt.com

- **Mobilenet_v2:**

- https://www.analyticsvidhya.com/blog/2023/12/what-is-mobilenetv2/?utm_source=chatgpt.com

- **Convolutional Neural Networks (CNNs):**

- https://www.analyticsvidhya.com/blog/2022/01/introduction-to-neural-networks/?utm_source=chatgpt.com

- **Overfitting and Regularization:**

- https://www.analyticsvidhya.com/blog/2018/11/neural-networks-hyperparameter-tuning-regularization-deeplearning/?utm_source=chatgpt.com

- **Optimizers:**

- https://www.analyticsvidhya.com/blog/2023/09/what-is-adam-optimizer/?utm_source=chatgpt.com

- Flask Basics: https://www.youtube.com/watch?v=lj4I_CvBnt0

Project Objectives

By the end of this project, you will: - Understand fundamental concepts and techniques used for Deep Learning in image classification - Gain comprehensive knowledge of agricultural disease datasets - Master data preprocessing and augmentation techniques for plant disease images - Learn to implement transfer learning using MobileNetV2 - Develop skills in model evaluation and performance optimization - Create a full-stack web application integrating the trained model - Deploy an AI-powered plant disease detection system

Project Flow:

The user interacts with the web interface following this flow:

1 .User Interface Interaction:

User accesses the web application and navigates to the upload page

2. Image Selection:

User selects or captures an image of a plant leaf

3. Image Processing:

The uploaded image is preprocessed and resized to match model requirements

4. Model Prediction:

The integrated MobileNetV2 model analyzes the image

5. Result Display:

Prediction results with disease classification and recommendations are displayed

Complete Project Activities:

1. Data Collection and Data Preprocessing:

- Download and organize the New Plant Diseases Dataset
- Data exploration and visualization
- Image augmentation (already applied in dataset)
- Dataset splitting into training and validation sets

2. Model Building

- Import necessary libraries
- Load pre-trained MobileNetV2 model
- Fine-tune the model architecture
- Configure model compilation
- Set up training callbacks

3. Model Training

- Train the model with data generators
- Monitor training progress
- Implement early stopping and learning rate reduction

4. Model Evaluation

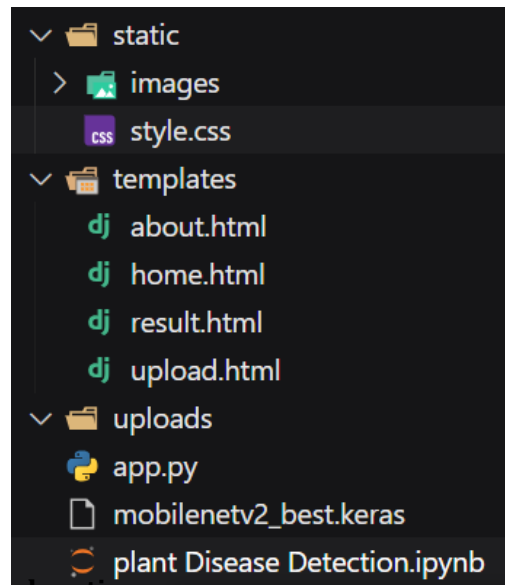
- Evaluate model performance on validation set
- Visualize training metrics
- Save the best performing model

5. Application Development

- Create HTML templates for web interface
- Develop Flask backend
- Integrate model with web application
- Test end-to-end functionality

Project Structure:

Create the Project folder which contains files as shown below



Project Structure Explanation

- static/
 - Contains static assets (CSS, JavaScript, uploaded images)
 - images/ – Stores user-uploaded images for display
 - style.css – Application styling
- templates/
 - HTML templates for Flask application
 - home.html – Landing page
 - about.html – Information about the technology
 - upload.html – Image upload interface
 - result.html – Disease prediction results
- uploads/
 - Temporary storage for file uploads
- app.py
 - Flask application backend
- mobilenetv2_best.keras
 - Trained model file
- plant Disease Detection.ipynb
 - Jupyter notebook with model training code

Milestone 1: Data Collection & Data Preprocessing

Activity 1.1: Download the Dataset

Dataset Information:

The **New Plant Diseases Dataset** is a comprehensive collection designed for **plant pathology research** and image-based **disease classification**.

- **Total Images:** 87,000+ augmented RGB images
- **Image Format:** JPEG
- **Number of Classes:** 38 (including healthy and diseased categories)
- **Plant Types:** Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Pepper, Potato, Raspberry, Soybean, Squash, Strawberry, Tomato
- **Image Size:** 256×256 pixels (*resized to 224×224 for MobileNetV2*)

Dataset Source

[Kaggle - New Plant Diseases Dataset](#)

Dataset Structure

```
new-plant-diseases-dataset/  
├── New Plant Diseases Dataset(Augmented)/  
│   ├── train/  
│   │   ├── Apple___Apple_scab/  
│   │   ├── Apple___Black_rot/  
│   │   ├── Apple___Cedar_apple_rust/  
│   │   ├── Apple___healthy/  
│   │   ├── ... (38 classes total)  
│   │   └── Tomato___healthy/  
│   └── valid/  
│       ├── Apple___Apple_scab/  
│       ├── ... (same 38 classes)  
│       └── Tomato___healthy/
```

Activity 1.1.1: Setting Up the Environment

Step 1: Upload Dataset to Google Colab

First, we need to set up our Kaggle API credentials to download the dataset directly in Colab:

```
# Upload your kaggle.json file  
from google.colab import files  
files.upload() # This will prompt you to upload kaggle.json
```

Choose Files kaggle.json
kaggle.json(application/json) - 70 bytes, last modified: 9/15/2025 - 100% done
Saving kaggle.json to kaggle.json

Activity 1.1.2: Importing Required Libraries

```
# Install required packages
!pip install -q kaggle "tensorflow>=2.17.0" gradio matplotlib

# Python imports & configuration
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Activity 1.1,3: Download and Extract Dataset

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# If dataset not downloaded yet, download & unzip (comment out if already present)
import os
if not os.path.exists("/content/new-plant-diseases-dataset"):
    !kaggle datasets download -d vip000ool/new-plant-diseases-dataset -p /content
    !unzip -q /content/new-plant-diseases-dataset.zip -d /content/new-plant-diseases-dataset

# Confirm dataset presence (small print)
!ls -lah /content/new-plant-diseases-dataset | sed -n '1,200p'

Dataset URL: https://www.kaggle.com/datasets/vip000ool/new-plant-diseases-dataset
License(s): copyright-authors
Downloading new-plant-diseases-dataset.zip to /content
 97% 2.61G/2.70G [00:27<00:02, 36.6MB/s]
100% 2.70G/2.70G [00:27<00:00, 107MB/s]
total 20K
drwxr-xr-x 5 root root 4.0K Oct 31 06:24 .
drwxr-xr-x 1 root root 4.0K Oct 31 06:23 ..
drwxr-xr-x 3 root root 4.0K Oct 31 06:23 new plant diseases dataset(augmented)
drwxr-xr-x 3 root root 4.0K Oct 31 06:23 New Plant Diseases Dataset(Augmented)
drwxr-xr-x 3 root root 4.0K Oct 31 06:24 test
```

Activity 1.2: Data Exploration and Visualization

Activity 1.2.1: Configure Dataset Paths

```
# User / experiment configuration
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 5
N_LAST_LAYERS = 10 # Unfreeze last 10 layers for fine-tuning
SEED = 1337

# Dataset paths
train_dir = "/content/new-plant-diseases-dataset/new plant diseases dataset(augmented)/New Plant Diseases Dataset(Augmented)/train"
valid_dir = "/content/new-plant-diseases-dataset/new plant diseases dataset(augmented)/New Plant Diseases Dataset(Augmented)/valid"

# Verify paths exist
for p in [train_dir, valid_dir]:
    if not os.path.exists(p):
        print(f"ERROR: path not found: {p}")
        sys.exit(1)

print("train_dir:", train_dir)
print("valid_dir:", valid_dir)

train_dir: /content/new-plant-diseases-dataset/new plant diseases dataset(augmented)/New Plant Diseases Dataset(Augmented)/train
valid_dir: /content/new-plant-diseases-dataset/new plant diseases dataset(augmented)/New Plant Diseases Dataset(Augmented)/valid
```

Activity 1.2.2: Visualize Sample Images

```

import random
from IPython.display import Image, display

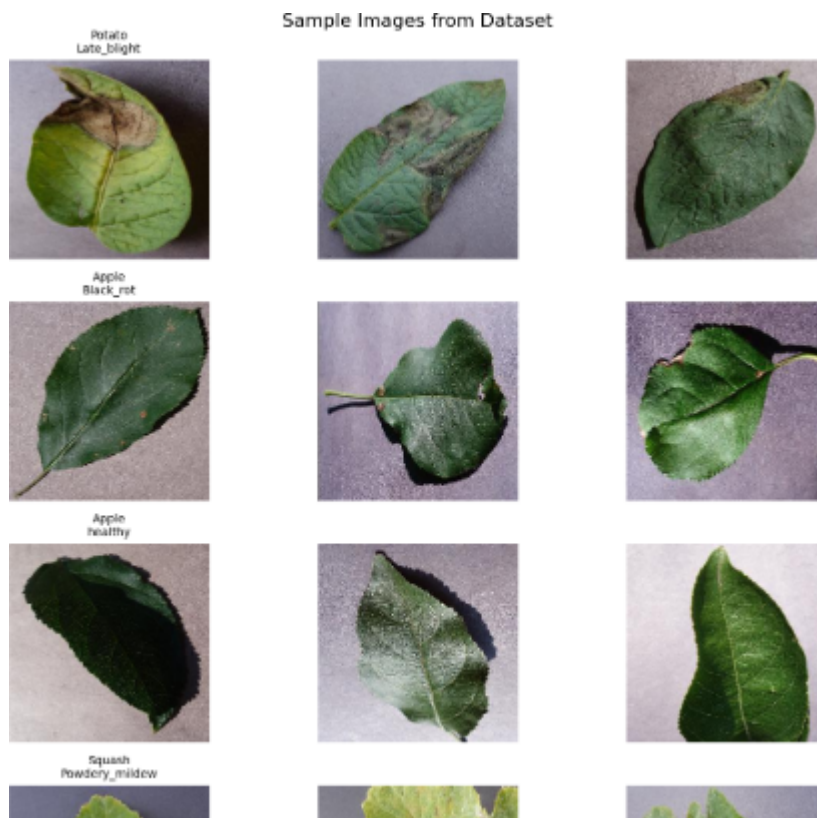
# Function to display random samples from each class
def show_sample_images(base_path, num_classes=5, images_per_class=3):
    """Display random sample images from the dataset"""
    class_names = os.listdir(base_path)
    random_classes = random.sample(class_names, min(num_classes, len(class_names)))

    fig, axes = plt.subplots(num_classes, images_per_class,
                             figsize=(12, 3*num_classes))
    fig.suptitle('Sample Images from Dataset', fontsize=16)

    for i, class_name in enumerate(random_classes):
        class_path = os.path.join(base_path, class_name)
        images = os.listdir(class_path)
        random_images = random.sample(images, min(images_per_class, len(images)))

        for j, img_name in enumerate(random_images):
            img_path = os.path.join(class_path, img_name)
            img = plt.imread(img_path)
            axes[i, j].imshow(img)
            axes[i, j].axis('off')
            if j == 0:
                axes[i, j].set_title(class_name.replace('_', '\n'), fontsize=10)

    plt.tight_layout()
    plt.show()
  
```



Activity 1.2.3: Dataset Statistics


```
# Count images in each split
def count_images(directory):
    """Count total images and images per class"""
    total_images = 0
    class_counts = {}

    for class_name in os.listdir(directory):
        class_path = os.path.join(directory, class_name)
        if os.path.isdir(class_path):
            num_images = len(os.listdir(class_path))
            class_counts[class_name] = num_images
            total_images += num_images

    return total_images, class_counts

# Get statistics
train_total, train_counts = count_images(train_dir)
valid_total, valid_counts = count_images(valid_dir)

print(f"Training images: {train_total}")
print(f"Validation images: {valid_total}")
print(f"Number of classes: {len(train_counts)}")
print(f"\nClass distribution (first 10):")
for i, (class_name, count) in enumerate(list(train_counts.items())[:10]):
    print(f"  {class_name}: {count} images")
```

Training images: 70295

Validation images: 17572

Number of classes: 38

Class distribution (first 10):

Corn_(maize)___healthy: 1859 images

Peach___healthy: 1728 images

Soybean___healthy: 2022 images

Tomato___Bacterial_spot: 1702 images

Pepper,_bell___healthy: 1988 images

Tomato___Late_blight: 1851 images

Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot: 1642 images

Apple___Apple_scab: 2016 images

Blueberry___healthy: 1816 images

Grape___Leaf_blight_(Isariopsis_Leaf_Spot): 1722 images

Activity 1.3: Data Preprocessing and Augmentation

Note: The dataset used in this project already contains augmented images. The original dataset was augmented using various techniques including rotation, flipping, and brightness adjustments. For this reason, we'll apply minimal additional augmentation during training.

Activity 1.3.1: Setup Data Generators

```
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

# Training data generator with light augmentation
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    horizontal_flip=True,
    rotation_range=20,
    zoom_range=0.15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    fill_mode='reflect'
)

# Validation data generator (no augmentation)
valid_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

# Create data generators
train_gen = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True,
    seed=SEED
)

valid_gen = valid_datagen.flow_from_directory(
    valid_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

Found 70295 images belonging to 38 classes.
Found 17572 images belonging to 38 classes.
```

Milestone 2: Model Building

Activity 2.1: Understanding Transfer Learning with MobileNetV2

What is Transfer Learning?

Transfer learning is a machine learning technique where a model developed for one task is reused as the starting point for a model on a second task. It's a popular approach in deep learning where pre-trained models are used as the starting point on computer vision and natural language processing tasks.

Why MobileNetV2?

MobileNetV2 is specifically designed for mobile and embedded vision applications. It features:

- Efficient Architecture: Uses depthwise separable convolutions
- Inverted Residuals: Linear bottleneck layers
- Lightweight: Fewer parameters compared to other architectures
- High Accuracy: Maintains competitive accuracy despite smaller size
- Pre-trained Weights: Trained on ImageNet with 1000 classes

Activity 2.2: Load Pre-trained MobileNetV2

```
from tensorflow.keras.applications import MobileNetV2

# Load pretrained MobileNetV2 backbone
base_model = MobileNetV2(
    input_shape=IMG_SIZE + (3,),
    include_top=False, # Exclude the final classification layer
    weights='imagenet' # Use pre-trained ImageNet weights
)

print("MobileNetV2 loaded successfully!")
print(f"Total layers in base model: {len(base_model.layers)}")

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet\_v2/mobilenet\_v2\_weights\_tf\_dim\_ordering\_tf\_kernels\_1.0\_224\_no\_top.h5
9406464/9406464 ————— 2s 0us/step
MobileNetV2 loaded successfully!
Total layers in base model: 154
```

Activity 2.3: Configure Transfer Learning Strategy

```
# Freeze all layers initially
for layer in base_model.layers:
    layer.trainable = False

# Unfreeze last N layers for fine-tuning
if N_LAST_LAYERS > 0:
    for layer in base_model.layers[-N_LAST_LAYERS:]:
        layer.trainable = True

# Count trainable parameters
trainable_count = sum([tf.size(w).numpy() for w in base_model.trainable_weights])
non_trainable_count = sum([tf.size(w).numpy() for w in base_model.non_trainable_weights])

print(f"Trainable parameters: {trainable_count:,}")
print(f"Non-trainable parameters: {non_trainable_count:,}")
print(f"Unfrozen last {N_LAST_LAYERS} layers for fine-tuning")
```

Activity 2.4: Build Complete Model Architecture

```
Trainable parameters: 732,480
Non-trainable parameters: 1,525,504
Unfrozen last 10 layers for fine-tuning
```

```
# Build the complete model
inputs = keras.Input(shape=IMG_SIZE + (3,))
x = base_model(inputs, training=False) # Frozen BatchNorm layers
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.35)(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.25)(x)
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)

model = keras.Model(inputs, outputs, name="mobilenetv2_plant_disease_classifier")

# Display model summary
model.summary()
```

Model: "mobilenetv2_plant_disease_classifier"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 256)	327,936
dropout_1 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 38)	9,766

Total params: 2,595,686 (9.90 MB)
Trainable params: 1,070,182 (4.08 MB)
Non-trainable params: 1,525,504 (5.82 MB)

Architecture Explanation:

1. Input Layer: Accepts 224x224x3 RGB images
2. MobileNetV2 Base: Pre-trained feature extractor (partially frozen)
3. GlobalAveragePooling2D: Reduces spatial dimensions to 1D
4. Dropout (0.35): Regularization to prevent overfitting
5. Dense (256): Fully connected layer with ReLU activation
6. Dropout (0.25): Additional regularization
7. Output Dense (38): Final classification layer with softmax

Milestone 3: Model Training

Activity 3.1: Compile the Model

```
# Compile model with appropriate settings
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4), # Low learning rate for fine-tuning
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Model compiled successfully!")
print(f"Optimizer: Adam (lr=1e-4)")
print(f"Loss function: Categorical Crossentropy")
print(f"Metrics: Accuracy")
```

Model compiled successfully!
Optimizer: Adam (lr=1e-4)
Loss function: Categorical Crossentropy
Metrics: Accuracy

Compilation Settings Explanation:

- **Optimizer:** Adam with learning rate 1e-4 (low rate prevents disrupting pre-trained weights)
- **Loss Function:** Categorical crossentropy (standard for multi-class classification)
- **Metrics:** Accuracy for monitoring training progress

Activity 3.2: Setup Training Callbacks

```
# Define training callbacks
callbacks = [
    # Save best model based on validation accuracy
    keras.callbacks.ModelCheckpoint(
        "/content/mobilenetv2_best.keras",
        monitor='val_accuracy',
        save_best_only=True,
        verbose=1
    ),

    # Reduce learning rate when validation loss plateaus
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        verbose=1
    ),

    # Stop training if no improvement
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=6,
        restore_best_weights=True,
        verbose=1
    )
]

print("Callbacks configured:")
print("  1. ModelCheckpoint - Saves best model")
print("  2. ReduceLROnPlateau - Adjusts learning rate")
print("  3. EarlyStopping - Prevents overfitting")
```

Callbacks configured:
1. ModelCheckpoint - Saves best model
2. ReduceLROnPlateau - Adjusts learning rate
3. EarlyStopping - Prevents overfitting

Callbacks Explanation:

1. **ModelCheckpoint:** Automatically saves the model with the best validation accuracy
2. **ReduceLROnPlateau:** Reduces learning rate by 50% if validation loss doesn't improve for 3 epochs
3. **EarlyStopping:** Stops training if validation loss doesn't improve for 6 epochs

Activity 3.3: Train the Model

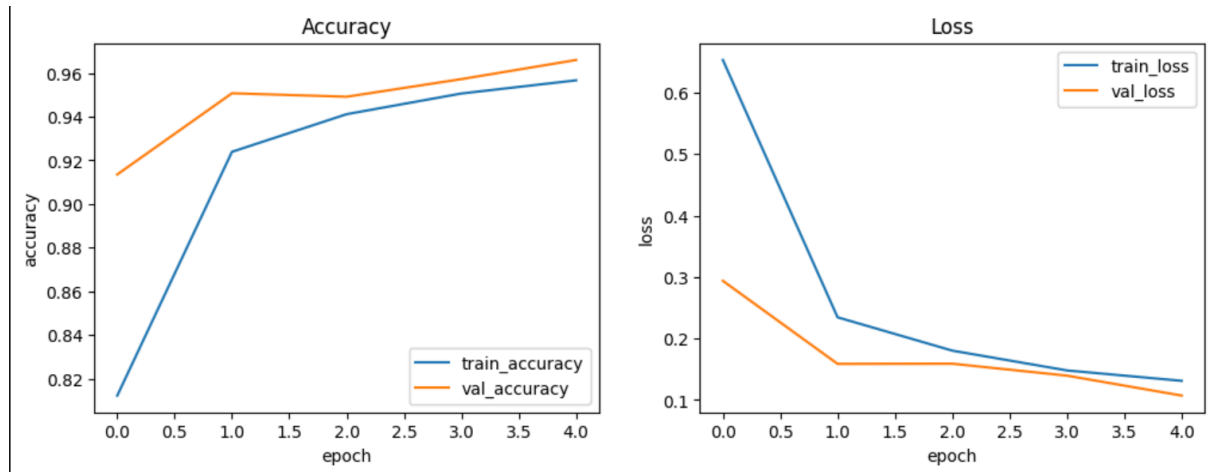
```
# ----- Train (single stage: some backbone layers trainable from start) -----
history = model.fit(
    train_gen,
    epochs=EPOCHS,
    validation_data=valid_gen,
    callbacks=callbacks
)

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().warn_if_super_not_called()`
Epoch 1/5
2197/2197 ----- 0s 383ms/step - accuracy: 0.6634 - loss: 1.2439
Epoch 1: val_accuracy improved from -inf to 0.91356, saving model to /content/mobilenetv2_best.keras
2197/2197 ----- 985s 402ms/step - accuracy: 0.6635 - loss: 1.2436 - val_accuracy: 0.9136 - val_loss: 0.2938 - learning_rate: 1.0000e-04
Epoch 2/5
2197/2197 ----- 0s 377ms/step - accuracy: 0.9190 - loss: 0.2525
Epoch 2: val_accuracy improved from 0.91356 to 0.95077, saving model to /content/mobilenetv2_best.keras
2197/2197 ----- 854s 389ms/step - accuracy: 0.9190 - loss: 0.2524 - val_accuracy: 0.9508 - val_loss: 0.1587 - learning_rate: 1.0000e-04
Epoch 3/5
2197/2197 ----- 0s 373ms/step - accuracy: 0.9389 - loss: 0.1887
Epoch 3: val_accuracy did not improve from 0.95077
2197/2197 ----- 843s 384ms/step - accuracy: 0.9389 - loss: 0.1887 - val_accuracy: 0.9492 - val_loss: 0.1590 - learning_rate: 1.0000e-04
Epoch 4/5
2197/2197 ----- 0s 373ms/step - accuracy: 0.9496 - loss: 0.1503
Epoch 4: val_accuracy improved from 0.95077 to 0.95726, saving model to /content/mobilenetv2_best.keras
2197/2197 ----- 846s 385ms/step - accuracy: 0.9496 - loss: 0.1503 - val_accuracy: 0.9573 - val_loss: 0.1396 - learning_rate: 1.0000e-04
Epoch 5/5
2197/2197 ----- 0s 376ms/step - accuracy: 0.9566 - loss: 0.1325
Epoch 5: val_accuracy improved from 0.95726 to 0.96603, saving model to /content/mobilenetv2_best.keras
2197/2197 ----- 851s 387ms/step - accuracy: 0.9566 - loss: 0.1325 - val_accuracy: 0.9660 - val_loss: 0.1072 - learning_rate: 1.0000e-04
Restoring model weights from the end of the best epoch: 5.
```

Activity 3.4: Visualize Training History

```
# ----- Plot training curves -----
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history.get('accuracy',[]), label='train_accuracy')
plt.plot(history.history.get('val_accuracy',[]), label='val_accuracy')
plt.xlabel('epoch'); plt.ylabel('accuracy'); plt.legend(); plt.title('Accuracy')

plt.subplot(1,2,2)
plt.plot(history.history.get('loss',[]), label='train_loss')
plt.plot(history.history.get('val_loss',[]), label='val_loss')
plt.xlabel('epoch'); plt.ylabel('loss'); plt.legend(); plt.title('Loss')
plt.show()
```



Training Analysis:

The training curves show:

- Rapid initial learning: Training accuracy jumps from 66% to 92% in the first epoch
- Steady improvement: Validation accuracy improves from 91% to 96.6% over 5 epochs
- No overfitting: Training and validation curves remain close, indicating good generalization
- Convergence: Both loss curves decrease smoothly, showing stable training

Milestone 4: Model Evaluation and Testing

Activity 4.1: Evaluate Model Performance

```
# ----- Evaluate -----  
val_loss, val_acc = model.evaluate(valid_gen)  
print(f"Validation loss: {val_loss:.4f}, accuracy: {val_acc:.4f}")
```

```
550/550 ----- 27s 49ms/step - accuracy: 0.9823 - loss: 0.0575  
Validation loss: 0.1072, accuracy: 0.9660
```

Activity 4.2 : Save the Final Model

```
# ----- Save final model -----  
final_path = "/content/mobilenetv2_final.keras"  
model.save(final_path)  
print("Saved final model to:", final_path)
```

```
Saved final model to: /content/mobilenetv2_final.keras
```


Milestone 5: Application Building

In this section, we will build a complete web application integrated with our trained MobileNetV2 model. The application provides an intuitive user interface where users can upload plant leaf images and receive instant disease detection results with actionable recommendations.

This milestone includes the following activities:

- Building HTML Templates
- Creating CSS Stylesheets
- Developing Flask Backend

Activity 5.1: Build Flask Backend (app.py)

File Location: `app.py` (in project root directory)

```
app.py 1 X
app.py > ...
82 @app.route('/predict', methods=['POST'])
83 def predict():
84     if 'file' not in request.files:
85         return jsonify({'error': 'No file uploaded'}), 400
86
87     file = request.files['file']
88
89     if file.filename == '':
90         return jsonify({'error': 'No file selected'}), 400
91
92     if file:
93         filename = secure_filename(file.filename)
94         filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
95         file.save(filepath)
96
97         try:
98             prediction = predict_image(filepath)
99
100             # Save image to static folder for display
101             static_filename = f"upload_{secrets.token_hex(8)}.jpg"
102             static_path = os.path.join(app.config['STATIC_FOLDER'], 'images', static_filename)
103             Image.open(filepath).save(static_path)
104
105             # Store in session
106             session['prediction'] = prediction
107             session['image_path'] = f'images/{static_filename}'
108
109             os.remove(filepath) # Clean up temp file
110
111             return jsonify({'success': True})
112         except Exception as e:
113             if os.path.exists(filepath):
114                 os.remove(filepath)
115             return jsonify({'error': str(e)}), 500
116
117 @app.route('/result')
118 def result():
119     prediction = session.get('prediction')
120     image_path = session.get('image_path')
121
122     if not prediction:
123         return redirect(url_for('upload'))
124
125     return render_template('result.html', prediction=prediction, image_path=image_path)
126
127 if __name__ == '__main__':
128     load_model()
129     app.run(debug=True, host='0.0.0.0', port=5000)
```

Run the code:

```

Debug mode: ON
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.68.69:5000
Press CTRL+C to quit
* Restarting with stat
Model loaded successfully from mobilenetv2_best.keras
* Debugger is active!
* Debugger PIN: 124-630-444
  
```

Activity 5.2: Building HTML Templates

For this project, we will create four HTML files that form the complete user interface. All HTML files should be saved in the templates folder.

Activity 5.2.1: Create Home Page (home.html)

The home page serves as the landing page for PlantCare AI, showcasing the application's features and capabilities.

File Location: [templates/home.html](#)

Key Features:

- Hero section with call-to-action
- Feature highlights (Instant Detection, High Accuracy, Easy to Use)
- Statistics display (38+ diseases, 95% accuracy, 10+ plant species)
- Responsive navigation bar

Code:

```

templates > dj about.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>About - Plant Disease Detection</title>
7    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
8  </head>
9  <body>
10   <!-- Navigation -->
11   <nav class="navbar">
12     <div class="nav-container">
13       <div class="nav-logo">
14         <div class="logo-icon">
15           <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fi
16             <path d="M11 20A7 7 0 0 1 9.8 6.1C15.5 5 17 4.48 19 2c1 2 2 4.18 2 8 0 5.5-4.78 1
17             <path d="M2 21c0-3 1.85-5.36 5.08-6C9.5 14.52 12 13 13 12"></path>
18           </svg>
19         </div>
20         <span class="logo-text">PlantCare AI</span>
21       </div>
22       <ul class="nav-menu">
23         <li><a href="/">Home</a></li>
24         <li><a href="/about" class="active">About</a></li>
25         <li><a href="/upload" class="btn-nav">Get Started</a></li>
26       </ul>
27     </div>
28   </nav>
29
30   <!-- About Hero -->
31   <section class="about-hero">
32     <div class="container">
33       <h1 class="page-title">About Our Technology</h1>
34       <p class="page-subtitle">Transforming agriculture with artificial intelligence</p>
35     </div>
36   </section>
37
38   <!-- Mission Section -->
39   <section class="mission-section">
40     <div class="container">
  
```

AI-Powered Plant Disease Detection

Protect your crops with cutting-edge artificial intelligence. Instant disease identification for healthier plants and better yields.

[Upload Image](#)[Learn More](#)

Why Choose PlantCare AI?



Instant Detection

Get results in seconds with our advanced AI model trained on thousands of plant images.



High Accuracy

Powered by MobileNetV2 architecture for precise disease identification across multiple plant species.



Easy to Use

Simple interface - just upload a photo and get instant diagnosis. No technical knowledge required.

38+

Plant Diseases

95%

Accuracy Rate

10+

Plant Species

24/7

Available

Ready to Protect Your Plants?

Start detecting plant diseases now with our AI-powered system

[Get Started Now](#)

About Plant Disease Detection System

Our advanced AI-powered platform utilizes state-of-the-art deep learning technology to accurately identify plant diseases in real-time. With a comprehensive database of plant conditions and diseases, we help farmers, gardeners, and agricultural professionals make informed decisions about plant health management.

**Smart Bridge Hyderabad**

Empowering agriculture through innovative AI solutions

© 2025 Smart Bridge Hyderabad. All rights reserved.

Activity 5.2.2: Create About Page (about.html)

The About page provides detailed information about the technology, mission, and AI model used in PlantCare AI.

File Location: <templates/about.html>

 PlantCare AI

HomeAboutGet Started

About Our Technology

Transforming agriculture with artificial intelligence

Our Mission

We are committed to revolutionizing plant health management through cutting-edge artificial intelligence. Our goal is to empower farmers, gardeners, and agricultural professionals with instant, accurate disease detection to protect crops and increase yields.

By making advanced AI technology accessible to everyone, we help prevent crop losses, reduce pesticide use, and promote sustainable farming practices.



How It Works

1

Upload Image

Take a photo of your plant leaf showing symptoms or upload an existing image.

2

AI Analysis

Our MobileNetV2 model processes the image using deep learning algorithms.

3

Get Results

Receive instant diagnosis with plant type and condition identification.

Our AI Model

Architecture

MobileNetV2 with transfer learning

Training Data

Thousands of labeled plant images

Supported Plants

Apple, Tomato, Potato, Grape, Corn, and more

Disease Coverage

38+ different diseases and healthy conditions



Early Detection

Identify diseases before they spread



Cost Effective

Reduce crop losses and treatment costs



Sustainable

Minimize unnecessary pesticide use



Fast & Accurate

Get results in seconds with high precision

Try It Now

Experience the power of AI-driven plant disease detection

[Start Detection](#)

About Plant Disease Detection System

Our advanced AI-powered platform utilizes state-of-the-art deep learning technology to accurately identify plant diseases in real-time. With a comprehensive database of plant conditions and diseases, we help farmers, gardeners, and agricultural professionals make informed decisions about plant health management.

 **Smart Bridge Hyderabad**
Empowering agriculture through innovative AI solutions
© 2025 Smart Bridge Hyderabad. All rights reserved.

Activity 5.2.3: Create Upload Page (upload.html)

The upload page allows users to select and upload plant leaf images for disease detection.

File Location: <templates/upload.html>

 PlantCare AI

HomeAboutGet Started

Upload Plant Image

Take or upload a clear photo of the affected plant leaf



Drag & Drop Image Here

or click to browse from your device

Supported formats: JPG, JPEG, PNG (Max 16MB)

Choose Image

Tips for Best Results

✓ Take photos in good lighting

✓ Focus on the affected leaf area

✓ Keep the camera steady

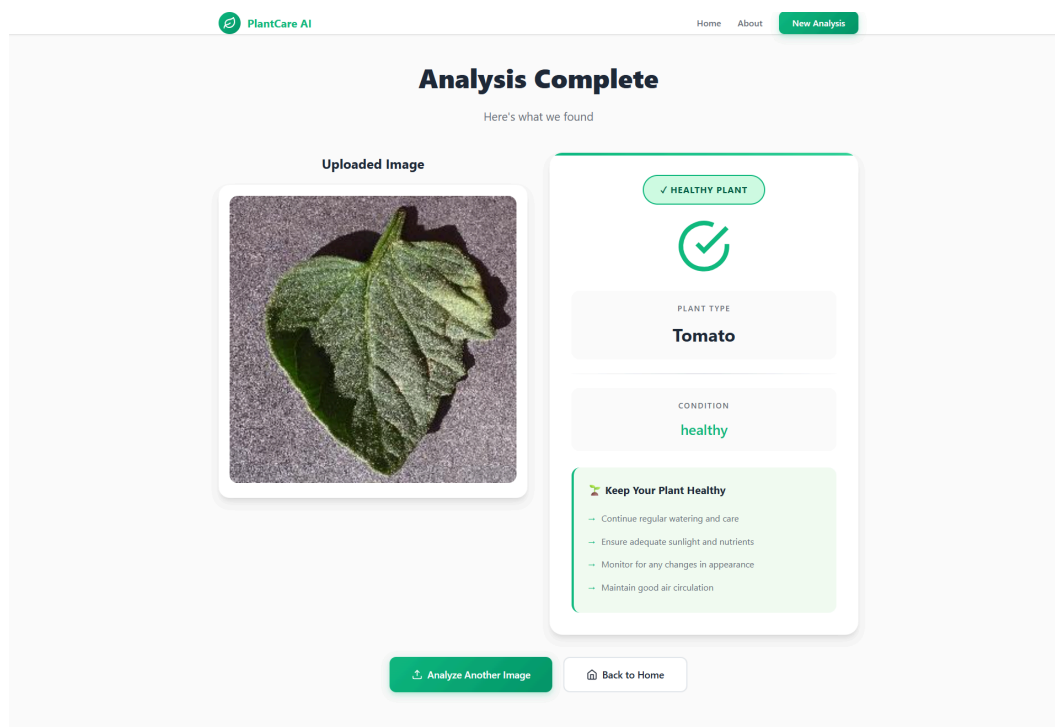
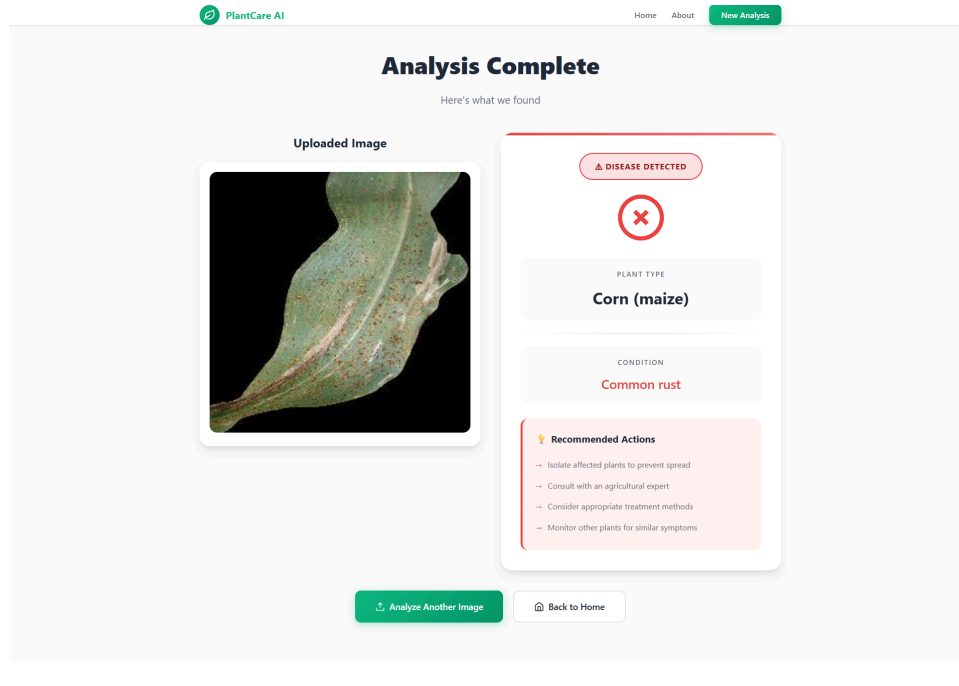
✓ Avoid blurry images

 Smart Bridge Hyderabad
Empowering agriculture through innovative AI solutions
© 2025 Smart Bridge Hyderabad. All rights reserved.

Activity 5.2.4: Create Result Page (result.html)

The result page displays the disease detection analysis with detailed information and recommendations.

File Location: <templates/result.html>



Future Implementations

Future plans for PlantCare AI focus on deployment and expansion:

Automated Agricultural Monitoring: Integrate the system into farm monitoring to enable real-time disease classification from captured images, automating reporting and intervention.

Mobile Application: Develop a user-friendly mobile app allowing home gardeners and small farmers to get instant disease analysis by simply taking a photo.

Educational Integration: Incorporate the model into agricultural training platforms for practical, hands-on learning in plant pathology.

Continuous Improvement: Expand the dataset to cover more diseases and plant species, and further fine-tune the MobileNetV2 architecture to ensure the highest possible accuracy across diverse environments.

Conclusion

The PlantCare AI project successfully developed an accurate and efficient AI-powered plant disease detection system. By leveraging the MobileNetV2 CNN through transfer learning, the model achieved a high validation accuracy of up to 96.6% using a dataset of over 87,000 images across 38 distinct disease classes.

The final product is a robust, full-stack web application (built with Flask) that seamlessly integrates the trained deep learning model into an intuitive user interface. This solution provides a reliable and scalable tool for precise and efficient disease identification, meeting the practical needs of farmers and gardeners.